
FROM PROMPT TO GOVERNED AGENCY

MCP, Dynamic Vaults, and an Auditable Architecture
for Path-Dependent Market Intelligence

CAPABILITY · MEMORY · GOVERNANCE · AUDITABILITY

"A trustworthy agentic system is not a model with more tools. It is a governed process in which purpose, capability, memory, time, evidence, and authority are explicitly connected."

ALEJANDRO REYNOSO

GOVERNANCE-FIRST AGENTIC SYSTEMS · 22 JULY 2026

Contents

Abstract	2
1. Introduction	2
2. Research Question and Contribution	3
3. Conceptual Foundations	4
3.1 From generative response to agentic process	4
3.2 Retrieval, tools, and persistent memory	4
3.3 Transparency, traceability, and auditability	4
4. The Classroom Experiment	5
4.1 Initial request	5
4.2 Data and truth-status design	5
4.3 Vault structure	5
5. Stage One: Direct Assistant-Mediated Execution	6
6. Stage Two: The Transparent Colab Notebook	6
7. Stage Three: Introducing MCP	7
7.1 What MCP is	7
7.2 What MCP is not	7
7.3 Why MCP changed the architecture	7
7.4 Tools, resources, and prompts in the case study	8
7.5 Lifecycle and connection boundary	8
8. The Complete Agentic Architecture	9
9. The Dynamic Vault	9
9.1 From repository to stateful memory	9
9.2 Path dependence	10
9.3 Memory selection and contamination risk	10
9.4 Human-readable and machine-readable duality	10
10. Orchestration: Supplying the Clock	10
11. Governance-First Control Model	11
11.1 Authorization envelope	11
11.2 Gates	11
11.3 Separation of duties	12
12. Audit Bundle and Manifest	12
13. Security and Failure Analysis	13
13.1 Tool misuse and excessive permission	13
13.2 Prompt injection through data or vault content	13
13.3 Data staleness and market-state ambiguity	13
13.4 Hallucinated or unsupported narrative	13
13.5 Silent partial failure	14
13.6 Audit-log tampering	14
14. Classroom Implementation and Validation	14
15. Production Migration	14

16. Pedagogical Interpretation	15
17. Broader Implications	15
18. Limitations	16
19. Conclusion	16
References	16
Appendix A. Minimal Tool Contract	17
Appendix B. Governed State Machine	17
Appendix C. Minimum Production Checklist	18

Abstract

A natural-language request can now initiate a consequential computational process: retrieve external data, compare it with prior observations, write persistent records, and synthesize a narrative. Yet the apparent simplicity of the request conceals a distributed architecture of data access, reasoning, memory, scheduling, authorization, and accountability. This paper reconstructs a classroom experiment that began with a request to observe five market indices once per minute, store sequential reports in an Obsidian vault on Google Drive, and produce an end-of-day narrative. Because the exercise compressed a market day into five minutes and direct Yahoo Finance retrieval was constrained, the implementation deliberately separated verified baseline context from simulated intraday states. The experiment then evolved through three stages: a direct assistant-mediated workflow, a transparent Colab notebook, and an auditable Model Context Protocol (MCP) architecture. The paper explains why MCP is not itself an agent, scheduler, database, or governance system. Rather, it is a standardized capability boundary through which an agent can discover and invoke bounded tools and access contextual resources. A full agentic system emerges only when MCP is combined with an orchestrator, a reasoning model, a dynamic vault, deterministic validation, authorization gates, provenance records, and human oversight. The resulting architecture turns a passive collection of files into a path-dependent institutional memory and transforms automation from an opaque convenience into an inspectable and contestable process.

Keywords: Model Context Protocol; agentic systems; Obsidian; dynamic vault; governance; auditability; provenance; market intelligence; human oversight; AI education.

1. Introduction

The most consequential change produced by contemporary language models is not simply that computers can generate fluent text. It is that ordinary language can increasingly function as an operational interface. A user can describe an objective rather than specify every computational step, and an AI system can interpret the objective, select tools, retrieve information, create artifacts, and revise its behavior in response to new evidence. This transition from language as description to language as delegation is the foundation of agentic computing.

The transition is also easy to misunderstand. A request such as “observe five market indices every minute, remember what happened before, and write the story of the day” sounds like a single instruction. Operationally, however, it contains several distinct responsibilities. The system must define the market universe, identify a data provider, retrieve observations, verify timestamps, distinguish live from delayed or simulated information, schedule repeated execution, consult prior reports, compute changes, generate interpretations, write persistent files, preserve lineage, enforce permissions, handle failures, and provide evidence of what occurred. None of these responsibilities disappears because the interface

is conversational.

This paper studies that hidden complexity through a deliberately compact classroom experiment. The initial objective was to create an Obsidian vault in Google Drive, observe five major equity indices at one-minute intervals, make each report use the reports that preceded it, and produce a final narrative that captured the intraday path rather than merely the closing state. A full trading day was compressed into five minutes so that students could observe an entire information cycle during one class. Five widely followed benchmarks formed the pedagogical universe: the S&P 500, Dow Jones Industrial Average, Nasdaq Composite, FTSE 100, and Nikkei 225.

The exercise exposed two immediate constraints. First, a conversational automation environment did not provide a one-minute recurrence mechanism; the clock had to be supplied by a separate orchestrator. Second, repeated direct access to Yahoo Finance was not a sufficiently dependable foundation for the demonstration. Consequently, the classroom run used verified market context as a baseline and five explicitly labelled simulated intraday states. That limitation became a methodological advantage. It forced a strict separation between observation, derivation, interpretation, and simulation, making visible a distinction that is frequently blurred in AI-generated financial commentary.

The work progressed through three levels of maturity. At the first level, the user asked the AI system to perform the task, and the system created the requested vault and reports. At the second level, a Colab notebook reconstructed every stage so that students could inspect configuration, transformations, predecessor retrieval, validation, narrative generation, file creation, hashing, and manifest construction. At the third level, the workflow was decomposed into an MCP-enabled architecture with bounded tools, an external clock, explicit authorization, a dynamic memory store, deterministic policy gates, and an audit bundle.

The paper advances four arguments. First, transparency and auditability are related but different. A notebook can reveal how a process is intended to work; an audit architecture must preserve evidence of how a particular execution actually worked. Second, MCP standardizes the interface between reasoning and capability, but it does not provide agency by itself. Third, an Obsidian vault becomes “dynamic” only when each cycle reads, evaluates, and extends its prior state under explicit lineage rules. Fourth, governance must be architectural rather than ornamental: permissions, classification, validation, failure handling, provenance, and human intervention must be designed into the execution path.

2. Research Question and Contribution

The central research question is:

How can a natural-language request for recurring, memory-aware market reporting be transformed into a transparent, governed, and auditable agentic system?

The question is deliberately broader than the specific market example. Market observations offer a useful teaching domain because timestamps, data provenance, path dependence, and the distinction between fact and interpretation are immediately meaningful. The same architecture can apply to legal case monitoring, tax-code change detection, portfolio research, compliance surveillance, scientific literature review, or operational incident management.

The paper makes five contributions. It first presents an end-to-end process model that connects user intent to tools, memory, outputs, and evidence. Second, it explains the precise role of MCP within that model and corrects the common misconception that MCP itself is an autonomous agent. Third, it defines a dynamic vault as a governed, append-oriented memory system whose records contain explicit predecessor relationships and provenance. Fourth, it develops a governance-first architecture in which failed controls generate visible evidence rather than silent omission. Finally, it proposes a pedagogical method in which students encounter agentic computing as a sequence of inspectable state transitions.

3. Conceptual Foundations

3.1 From generative response to agentic process

A generative model maps context into an output. An agentic system adds a loop. It interprets an objective, determines what information or action is required, invokes capabilities, observes the result, updates its working state, and continues until a stopping condition is reached. Research such as ReAct demonstrated the value of interleaving reasoning and action: actions connect the model to external sources, while observations allow the reasoning process to revise its plan [1].

For governance purposes, agency should not be defined by anthropomorphic language. A system is agentic to the extent that it has bounded discretion over a sequence of state-dependent actions. This definition has three advantages. It recognizes degrees of agency; it focuses attention on decisions and permissions; and it distinguishes autonomy from mere scheduling. A cron job that downloads a fixed file is automated but minimally agentic. A model that chooses among tools based on evolving evidence exercises greater discretion. A system that can write, notify, transact, or delete introduces still more consequential agency and therefore requires stronger controls.

The classroom system used constrained agency. The universe, number of cycles, data classification, output locations, and validation rules were fixed. The reasoning layer was permitted to compare states and formulate a narrative, but it was not permitted to trade, change the approved universe, relabel simulated data, or suppress a failed gate. This illustrates a critical design principle: useful agentic behavior does not require unlimited autonomy.

3.2 Retrieval, tools, and persistent memory

Retrieval-augmented generation introduced a widely used pattern in which external documents are retrieved and supplied as context before generation [2]. MCP is broader. Retrieval is one possible tool behavior; MCP can also expose computation and controlled actions. A server may offer a resource containing a prior report, a tool that validates a snapshot, or a tool that writes an approved artifact. Thus, the relevant distinction is not “RAG or MCP.” A governed architecture may use retrieval inside an MCP-exposed capability.

Memory also has multiple forms. The model’s context window provides temporary working memory. A database or vector index supports queryable machine memory. A vault provides persistent, human-readable documentary memory. In the experiment, the vault was chosen because every report could be opened, linked, inspected, and taught without requiring a specialized interface. Its Markdown files served simultaneously as operational records, teaching objects, and audit evidence.

3.3 Transparency, traceability, and auditability

Transparency answers: “Can an informed observer understand the design?” Traceability answers: “Can an observer follow the relationships among inputs, transformations, decisions, and outputs?” Auditability answers: “Does sufficient durable evidence exist to reconstruct and evaluate a particular execution against defined criteria?”

These properties form a progression but are not substitutes for one another. Source code can be transparent while an execution remains unauditible because inputs were not retained. Logs can be detailed yet untrustworthy if they are editable without detection. A hash can establish file integrity but not the truth of the data inside the file. A human-readable report can communicate conclusions while omitting the machine-readable evidence needed for reproduction.

The W3C PROV model is useful here because it conceptualizes provenance through entities, activities, and agents and the relations among them [3]. In the market system, a snapshot is an entity; validation and synthesis are activities; the data adapter, orchestrator, model, and authorized user are agents in the provenance sense. The audit bundle captures their relationships.

4. The Classroom Experiment

4.1 Initial request

The operational request contained six requirements:

1. Create an Obsidian vault in a specified Google Drive working directory.
2. Observe five major market indices once per minute.
3. Produce a report after each observation.
4. Make every new report use information from its predecessors.
5. At the end of the sequence, generate a narrative of the intraday dynamics.
6. Compress the day into five minutes for classroom demonstration.

The final requirement transformed the exercise from live monitoring into a simulation of temporal process. The goal was not to pretend that five minutes were a real trading day. It was to expose the architecture of repeated observation, memory retrieval, state comparison, and cumulative synthesis within a teachable interval.

4.2 Data and truth-status design

Four truth statuses were defined:

Status	Meaning	Permitted use
VERIFIED	Independently retrieved baseline fact with source and timestamp	Context and factual reporting
DERIVED	Reproducible calculation from stored inputs	Metrics and comparisons
INTERPRETATION	Analytical judgment grounded in identified evidence	Narrative, visibly separated from facts
SIMULATED	Pedagogical scenario value, not a live quotation	Classroom dynamics only

This taxonomy was not cosmetic metadata. A report could not pass the classification gate unless every market value carried a status. The final narrative could discuss the simulated path, but it could not represent that path as an observed historical day. If a live adapter were later activated, the taxonomy could be expanded to LIVE, DELAYED, STALE, and UNAVAILABLE.

The five scenario states were designed to create a path with analytical content: an opening shock, a broader early selloff, a reversal, regional or sectoral divergence, and a differentiated close. The endpoint alone could not communicate that sequence. The final report therefore had to use the entire chain:

$$H_5 = \{S_1, S_2, S_3, S_4, S_5\},$$

where S_t is the governed snapshot at cycle t and H_t is the admissible history available to the agent.

4.3 Vault structure

The first vault used four functional areas:

Folder	Purpose
00_Governance	Methodology, scope, classifications, and control rules
01_Snapshots	Sequential minute reports and predecessor links
02_Daily_Narratives	End-of-sequence synthesis
03_Data	Structured observations in CSV or JSON
04_Audit	Run manifest, events, hashes, and validation results

This structure separates policy, evidence, interpretation, and audit. Obsidian links make lineage navigable to a person; identifiers and hashes make it verifiable by a program. Neither replaces the other.

5. Stage One: Direct Assistant-Mediated Execution

At the first stage, the user delegated the outcome in natural language. The assistant interpreted the objective, selected a reasonable index universe, created a folder structure, generated sequential reports, linked each report to earlier reports, and wrote an end-of-day narrative. From the user's perspective, the interaction was admirably simple. That simplicity is an important feature of language-first computing.

Yet the stage exposed an epistemic asymmetry. The user could inspect the outputs but could not easily distinguish every internal transformation that produced them. Several questions remained: Which exact predecessor files were consulted? What validation occurred before writing? How were simulated values separated from retrieved facts? What happened if a value was missing? Were hashes calculated before or after the manifest? Did the system retry a failed retrieval? Was a report regenerated?

The problem was not that the outputs were necessarily wrong. The problem was that outcome inspection alone is an inadequate governance mechanism. The natural-language interface compressed not only technical complexity but also evidence about the process. This motivated the transparent notebook.

6. Stage Two: The Transparent Colab Notebook

The notebook translated the apparently singular action into an inspectable pipeline:

Configuration → Source Register → Observation → Validation
→ Memory Retrieval → Synthesis → Persistence → Manifest.

The notebook contained narrative cells that explained each responsibility and executable cells that implemented it. Its design followed five pedagogical principles.

First, configuration was explicit. The index universe, cycle count, classification mode, folder paths, authorization token or run identifier, and freshness thresholds were visible rather than embedded invisibly in prose.

Second, intermediate states were materialized. Students could see the structured snapshot before reading the prose report. They could compare the current observation with the prior one and reproduce derived changes.

Third, memory retrieval was observable. Before generating report R_t , the notebook enumerated the prior reports supplied to the synthesis function. A lineage assertion required:

$$P_t = \{R_1, \dots, R_{t-1}\}, \quad t > 1,$$

where P_t is the predecessor set recorded for report t . For the first report, $P_1 = \emptyset$ was valid and explicitly documented.

Fourth, controls failed visibly. Validation cells raised errors if classification labels, timestamps, required fields, predecessor links, or output artifacts were absent. A transparent workflow should not convert a failure into a plausible paragraph.

Fifth, the notebook ended with evidence. It generated SHA-256 digests, a run manifest, and integrity checks. The result was not just a set of reports but a demonstrable relationship between configuration, inputs, transformations, and outputs.

The notebook increased transparency and reproducibility, but it did not fully solve modularity or operational integration. Its functions still lived inside one executable document, and external capabilities were not yet exposed through a standardized contract. That limitation motivated MCP.

7. Stage Three: Introducing MCP

7.1 What MCP is

The Model Context Protocol is an open protocol for connecting AI applications to external context and capabilities. Its architecture separates a host application, one or more clients, and one or more servers. At the protocol level, clients and servers communicate using JSON-RPC, negotiate capabilities, and may expose primitives including tools, resources, and prompts [4–6].

In an OpenAI application architecture, an MCP server can define tools, enforce authentication, return structured data, and optionally connect tool responses to an interface; the model determines when to call the tools and explains the results [7]. This division is crucial. MCP provides a grammar of interaction, not an all-purpose runtime.

7.2 What MCP is not

MCP is not:

- ▶ the language model that reasons;
- ▶ the scheduler that creates one-minute triggers;
- ▶ the market-data provider;
- ▶ the vault or database;
- ▶ the policy authority that grants permission;
- ▶ the audit system that establishes completeness;
- ▶ the human accountable for deployment.

The concise formulation used in the experiment was:

MCP provides standardized hands and contextual access; the orchestrator provides the clock; the model provides bounded reasoning; the vault provides durable memory; governance determines what may happen; the audit layer preserves evidence of what did happen.

7.3 Why MCP changed the architecture

Before MCP decomposition, a notebook function might directly call a provider, write a file, and return prose. After decomposition, capabilities became named contracts with typed inputs and structured outputs. The model did not need provider-specific knowledge. It could call `get_market_snapshot`; an adapter beneath that tool could use a classroom simulator, `yfinance`, or a licensed provider.

This abstraction reduces coupling. It also makes governance more tractable because permissions can attach to capabilities. Reading prior reports and publishing a narrative need not carry the same autho-

rization. A read-only MCP server may expose resources without exposing write tools. A production system can require explicit approval for high-impact operations.

7.4 Tools, resources, and prompts in the case study

The MCP design mapped primitives to responsibilities:

Primitive	Example	Governance meaning
Tool	<code>get_market_snapshot</code>	Bounded data retrieval with a declared schema
Tool	<code>validate_snapshot</code>	Deterministic policy evaluation
Tool	<code>save_report</code>	Controlled mutation of the vault
Tool	<code>verify_vault_integrity</code>	Independent post-write check
Resource	prior report or policy document	Context that can be read without implying mutation
Prompt	market synthesis template	Reusable instruction pattern, not an authorization grant

Eleven governed tools were included in the classroom package, covering universe discovery, snapshot retrieval, market status and provider metadata, validation, predecessor retrieval, snapshot and report persistence, link creation, daily finalization, manifest writing, and vault verification. The precise number is less important than the separation of responsibilities.

7.5 Lifecycle and connection boundary

MCP defines a lifecycle with initialization, capability negotiation, normal operation, and shutdown [6]. In a local classroom package, a client can communicate with a local server process. In a production environment, a remote MCP server requires a reachable endpoint, transport security, authentication, scoped authorization, and operational monitoring. Capability discovery should never be interpreted as permission to use every discovered capability. Discovery answers “what exists”; authorization answers “what this principal may invoke for this purpose.”

8. The Complete Agentic Architecture

The resulting architecture contains eight separable layers.

Layer	Primary responsibility	Typical failure if omitted
Human intent	Define purpose, scope, and acceptable outcomes	Goal ambiguity
Orchestrator	Trigger cycles, manage state, retries, and stopping	No reliable recurrence
Reasoning agent	Select permitted tools and synthesize evidence	Mechanical output without adaptation
MCP capability layer	Standardize tools/resources and their schemas	Brittle point-to-point integration
Data adapters	Connect to simulator, Yahoo-derived library, or licensed feed	Provider lock-in or inconsistent fields
Dynamic vault	Persist observations, reports, lineage, and policies	Stateless commentary
Governance engine	Enforce authorization, freshness, classification, and publication rules	Unsafe or misleading execution
Audit layer	Preserve events, manifests, hashes, and verification results	Unreconstructable history

The control flow for a single cycle is:

1. The orchestrator creates a cycle identifier and records the trigger.
2. The authorization gate verifies the actor, purpose, universe, mode, and allowed actions.
3. The agent or deterministic controller requests the approved universe.
4. The market tool retrieves a structured snapshot through the configured adapter.
5. The validation tool evaluates completeness, freshness, classification, and consistency.
6. The memory tool retrieves the admissible predecessor set from the vault.
7. The reasoning layer compares the current snapshot with the historical path.
8. The narrative is checked for unsupported claims and classification leakage.
9. The write tool persists structured data and the human-readable report.
10. The integrity tool reads the persisted artifacts back and verifies links and hashes.
11. The audit layer records the complete decision and result.

The fifth cycle triggers an additional synthesis over the complete admissible history. Importantly, the final narrative is generated from stored evidence, not from an unrecorded conversational memory.

9. The Dynamic Vault

9.1 From repository to stateful memory

A conventional vault is a collection of documents. A dynamic vault participates in the control loop. At time t , it stores the prior state, supplies selected context to the current cycle, receives a new governed record, and changes the evidence available at $t + 1$. Formally:

$$V_t = V_{t-1} \cup \{S_t, R_t, A_t\},$$

where V_t is vault state, S_t is the structured snapshot, R_t is the report, and A_t is the audit event. The transition is permitted only if all mandatory gates return PASS.

The vault therefore has two simultaneous roles. It is an epistemic memory, containing what the system knows and how interpretations evolved. It is also an accountability record, containing the evidence needed to challenge those interpretations.

9.2 Path dependence

The distinguishing feature of the experiment is path-dependent narrative. A report at time t is a function not only of S_t but of an admissible historical sequence:

$$R_t = f(S_t, H_{t-1}, C, \Pi),$$

where C is the fixed configuration and Π is the policy set. The final narrative is:

$$N = g(H_T, \Delta H_T, E, \Pi),$$

where ΔH_T denotes state transitions and E denotes external baseline evidence.

This matters because two days can close at the same level and tell entirely different stories. One may be stable throughout; another may involve a sharp selloff and recovery. A snapshot-only architecture loses this temporal structure. The dynamic vault preserves it.

9.3 Memory selection and contamination risk

More memory is not always better. Supplying every vault file to every cycle can introduce stale evidence, contradictory policy versions, irrelevant documents, or prompt injection contained in stored text. Memory retrieval must therefore be governed by scope, time, document type, trust status, and policy version.

The classroom rule was intentionally simple: report t consulted all accepted reports 1 through $t - 1$. A production system might use a tiered policy: current-day accepted snapshots, the latest approved methodology, selected comparison days, and no untrusted free-form notes. Every retrieved item should be listed in the audit event.

9.4 Human-readable and machine-readable duality

Markdown links are excellent for explanation but insufficient for machine assurance. Conversely, JSON lineage is excellent for validation but less intuitive for students. The architecture uses both. Each report contains readable links such as `[[Minute_03_Reversal]]`; the manifest records identifiers, predecessor arrays, timestamps, content hashes, and policy versions.

10. Orchestration: Supplying the Clock

An MCP server is reactive: it waits for a client request. It does not independently decide that another minute has elapsed. The orchestrator supplies recurrence and manages the run lifecycle.

In the classroom implementation, a controlled loop executed five cycles with an optional sixty-second interval. For demonstration and testing, the interval could be shortened or skipped while logical timestamps remained explicit. A production orchestrator would add market calendars, time-zone handling, idempotency keys, distributed locks, retry policies, timeouts, health checks, and alerting.

Idempotency is particularly important. If a network timeout occurs after a report is written but before success is acknowledged, an ungoverned retry may create duplicate reports. Each cycle should therefore

have a unique key:

$$K_t = \text{hash}(\text{run-id}, \text{cycle}, \text{scheduled-time}, \text{universe-version}).$$

The persistence layer rejects a second write with the same key unless an authorized correction workflow creates a new version. “Exactly once” is rarely guaranteed by a scheduler alone; it is approximated through idempotent operations and durable state.

The orchestrator also defines terminal conditions. The run ends after five accepted classroom cycles, after the scheduled market close, after an authorized cancellation, or after a fail-closed event that policy defines as non-recoverable. It should never continue indefinitely merely because the model can generate another action.

11. Governance-First Control Model

The architecture aligns naturally with the NIST AI Risk Management Framework’s cross-cutting GOVERN function and its MAP, MEASURE, and MANAGE functions [8]. Governance is not a final review step. It establishes roles, policies, tolerances, and accountability before execution and operates throughout the lifecycle.

11.1 Authorization envelope

Every run should have an authorization envelope containing:

- ▶ principal or initiating user;
- ▶ declared purpose;
- ▶ approved data universe;
- ▶ provider and mode;
- ▶ permitted read and write tools;
- ▶ destination scope;
- ▶ recurrence and expiration;
- ▶ approval requirements;
- ▶ maximum impact;
- ▶ policy version.

In the classroom package, a deliberately incorrect authorization was tested and rejected. This negative test was pedagogically important. A system has not demonstrated access control merely because an authorized request succeeds; it must also demonstrate that an unauthorized request fails visibly.

11.2 Gates

The core gates were:

Gate	Question	Failure behavior
Authorization	Is the run and requested operation permitted?	Reject before external action
Universe	Are all symbols approved and versioned?	Reject unknown symbols
Provenance	Is source, adapter, retrieval time, and mode recorded?	Quarantine observation
Freshness	Is each value within the applicable age threshold?	Mark stale or reject publication

Gate	Question	Failure behavior
Classification	Is each value correctly labelled?	Reject report generation
Completeness	Are required fields present?	Reject or explicitly degrade
Consistency	Are calculations reproducible and values plausible?	Flag anomaly and require review
Memory	Was the correct predecessor set retrieved?	Reject cumulative claim
Narrative	Are fact, derivation, interpretation, and simulation separated?	Withhold publication
Persistence	Were files written to the authorized location?	Record failure and retry safely
Integrity	Do links, counts, schemas, and hashes verify?	Mark run incomplete

Some failures should be fail-closed; others may support controlled degradation. If the FTSE is closed while US markets are open, “closed” may be a valid status, not missing data. If a live quote becomes stale, the report may be published only if the stale condition is conspicuous and policy allows it. The key is that failure semantics are decided by policy rather than improvised by the model.

11.3 Separation of duties

The same component should not unilaterally retrieve, interpret, approve, publish, and certify its own output in high-impact environments. Even within a compact system, logical separation is possible:

- ▶ adapters retrieve data;
- ▶ deterministic validators assess schema and policy;
- ▶ the model produces bounded interpretation;
- ▶ a publication controller checks required evidence;
- ▶ the integrity verifier confirms persistence;
- ▶ a human approves exceptional or high-impact actions.

This does not eliminate model risk. It contains it within a larger control system.

12. Audit Bundle and Manifest

The audit bundle is the durable representation of an execution. It should contain at least:

1. the declared objective and authorization envelope;
2. configuration and policy versions;
3. tool inventory and schemas;
4. provider metadata and source register;
5. raw or normalized observations, subject to licensing constraints;
6. validation results for each gate;
7. predecessor sets and retrieved resource identifiers;
8. generated reports and final narrative;
9. tool-call events with timestamps and statuses;
10. errors, retries, overrides, and approvals;
11. artifact hashes and a manifest;
12. final integrity-verification result.

A simplified audit event is:

```
{
  "run_id": "market-classroom-2026-07-22",
  "cycle": 3,
  "tool": "save_report",
  "authorization": "PASS",
  "classification": "SIMULATED",
  "predecessors": ["minute-01", "minute-02"],
  "policy_version": "1.0",
  "status": "ACCEPTED",
  "output_id": "minute-03",
  "sha256": "<digest>",
  "timestamp_utc": "<ISO-8601>"
}
```

Hashes establish whether bytes have changed since hashing; they do not prove that the source was accurate, the policy appropriate, or the interpretation sound. Auditability is therefore layered: provenance supports attribution, validation supports conformance, hashes support integrity, and human review supports accountability.

The manifest should itself be versioned. If it contains hashes of other artifacts, a root manifest hash can be recorded outside the mutable bundle or in an append-only store. A classroom demonstration can stop at local verification; a production system may require signed attestations, immutable object storage, centralized logging, and retention policies.

13. Security and Failure Analysis

Connecting a model to external systems increases capability and attack surface simultaneously. The main risks in this architecture include:

13.1 Tool misuse and excessive permission

A broadly scoped write tool could overwrite unrelated Drive files. Mitigations include least-privilege credentials, destination allowlists, append-oriented writes, schema validation, explicit tool annotations, and approval gates for destructive or external-impact actions.

13.2 Prompt injection through data or vault content

A retrieved webpage or Markdown note may contain instructions that attempt to redirect the agent. External content must be treated as data, not authority. System policy and tool authorization must take precedence over content inside retrieved resources. Memory selection should exclude untrusted instructions, and high-impact tool inputs should be validated outside the model.

13.3 Data staleness and market-state ambiguity

Global indices operate in different time zones. A value can be current relative to the last close yet unsuitable for an “intraday” comparison. The tool response should include exchange time, retrieval time, market status, currency, and provider status. Freshness rules must be market-aware.

13.4 Hallucinated or unsupported narrative

Even with correct data, a model may infer causality without evidence. The narrative policy should distinguish observed co-movement from causal explanation. Claims about causes require identified external evidence; otherwise they must be labelled hypotheses or omitted.

13.5 Silent partial failure

A report covering four of five indices may look complete. Structured completeness checks should precede prose generation, and unavailable values should remain explicit. Fluent text must never function as an error mask.

13.6 Audit-log tampering

If the same process can rewrite its audit record, the evidentiary value is limited. Append-only storage, remote log collection, signed events, restricted correction workflows, and independent verification strengthen assurance.

14. Classroom Implementation and Validation

The final package combined an executable Python MCP server, the governed tool set, a five-cycle orchestrator, simulated and optional market-data adapters, a Drive-compatible Obsidian vault generator, policy checks, cumulative reports, structured observations, audit events, a manifest, integrity tests, connection instructions, and a transparent thirty-cell Colab notebook.

The deterministic classroom mode required no model API credential. This design choice was educationally important. Students could execute the state machine, inspect the gates, and verify the audit bundle before enabling any optional model-based synthesis. It separated the architecture of accountable action from the probabilistic behavior of a language model.

The completed validation demonstrated:

- ▶ rejection of an incorrect authorization;
- ▶ five accepted governed cycles;
- ▶ correct predecessor lineage for every report;
- ▶ consistent SIMULATED classification;
- ▶ an end-of-sequence narrative based on the full path;
- ▶ successful artifact hashing and verification;
- ▶ syntactic validation of all notebook code cells.

These tests do not prove production readiness. They prove that the pedagogical claims are executable. The distinction matters. A classroom prototype validates concepts under bounded conditions; production assurance requires provider contracts, real authentication, observability, adversarial testing, operational ownership, and incident procedures.

15. Production Migration

A production implementation should preserve the logical interfaces while replacing several components.

First, the simulated adapter should be replaced with a licensed market-data feed appropriate to the use case. `yfinance` can be useful for education and research prototypes but is an unofficial interface and should not be treated as a contractual production feed. The normalized MCP response schema should remain stable so that the agent does not depend on provider-specific fields.

Second, Colab should be replaced by a persistent execution environment. A managed scheduler or event-driven service should run the orchestrator with monitored service identities, secret management, health checks, and retry queues.

Third, local or Drive-only audit evidence should be supplemented with centralized, access-controlled, append-oriented logs. Retention and deletion must reflect legal, regulatory, contractual, and privacy requirements.

Fourth, authentication and authorization should be implemented end to end. Remote MCP endpoints require secure transport and appropriate authentication; tool-level authorization must still evaluate purpose and scope. A valid connection is not equivalent to permission for every action.

Fifth, model behavior should be evaluated using representative and adversarial cases: missing markets, conflicting sources, market holidays, corporate actions, extreme moves, stale values, poisoned notes, duplicate triggers, provider outages, and ambiguous causal news.

Sixth, human oversight should be calibrated to impact. A descriptive internal briefing may permit automatic publication after deterministic checks. A client communication, regulatory filing, or trading action should require stronger approvals and possibly independent review. Escalation thresholds must be explicit.

16. Pedagogical Interpretation

The exercise offers a “glass-box progression” for teaching AI systems.

At the first level, students see the power of natural-language delegation. A simple request produces a useful artifact. At the second level, the notebook slows the process down and makes every transformation visible. At the third level, MCP separates reasoning from capabilities and introduces a protocol boundary. At the fourth level, governance gates reveal that not every technically possible action is authorized. At the fifth level, the audit bundle allows students to reconstruct a run and challenge its claims.

This sequence avoids two common pedagogical failures. The first is presenting AI as magic, in which outputs appear without architecture. The second is presenting AI as only code, in which the conceptual transformation is lost in implementation details. The staged method begins with experience, moves to explanation, and ends with accountable system design.

The dynamic vault is particularly valuable in education because it makes temporal reasoning tangible. Students can open minute one, follow links to later reports, inspect how an early hypothesis was confirmed or rejected, and compare the final narrative with the structured observations. The vault becomes a laboratory notebook for an artificial agent.

The deliberate failed-authorization test is equally important. Governance is often taught as policy prose. Here, governance is executable: an unauthorized run produces a rejected event. Students learn that a control is meaningful only when it changes system behavior and leaves evidence.

17. Broader Implications

The architecture suggests a more general model of organizational AI. An enterprise does not need one omnipotent autonomous agent. It needs a federation of bounded capabilities, governed memory, explicit orchestration, and accountable human ownership. MCP can reduce the cost of connecting models to those capabilities, but organizational intelligence arises from the quality of the complete system.

The dynamic vault also points toward institutional memory that is both machine-actionable and human-legible. Traditional knowledge bases often preserve conclusions without the path that produced them. Conversational systems often preserve dialogue without validated operational state. A governed vault can preserve observation, interpretation, decision, and lineage together.

This has implications for finance. Market intelligence is not merely a stream of prices or a daily paragraph. It is a sequence of evidence-conditioned hypotheses. A system that records how interpretations evolve can support post-event review: Which signal changed the narrative? Which assumption persisted too long? Which missing observation should have triggered caution? These questions are valuable even when the system never executes a trade.

The same pattern applies to other domains. In tax planning, the dynamic vault can link quarterly

code changes to prior structures and approvals. In civil litigation, it can connect new filings to prior strategy notes while preserving confidentiality and authorization. In scientific research, it can relate new papers or results to evolving hypotheses. The domain changes; the architecture of governed observation, memory, action, and evidence remains recognizable.

18. Limitations

The experiment has important limitations. The five-cycle “day” is a pedagogical compression, not a realistic model of microstructure or global market synchronization. Simulated states validate workflow behavior, not market-data accuracy. The index universe is illustrative rather than analytically optimal. The narrative layer was constrained and cannot establish causality from prices alone. File hashes do not create immutable provenance. A Drive-backed Markdown vault is accessible and teachable but may not meet enterprise requirements for concurrency, query performance, retention, or access control. Finally, protocol conformance does not guarantee secure server implementation; every MCP server and underlying adapter must still be threat-modelled and operated responsibly.

These limitations do not weaken the central lesson. They clarify the boundary between a conceptual prototype and a production system—precisely the distinction that governance-first design is intended to make visible.

19. Conclusion

The experiment began with a request that sounded simple: watch markets, remember the past, and tell the story of the day. Its value lay in revealing that the request was not simple at all. It was a compressed specification for a socio-technical system involving data, time, memory, reasoning, permissions, persistence, and accountability.

The first implementation demonstrated the expressive power of natural-language delegation. The notebook made the internal workflow transparent. The MCP architecture modularized external capabilities and context. The orchestrator supplied recurrence. The dynamic vault made each report part of an evolving, inspectable memory. Governance gates constrained what the system could accept and publish. The audit bundle preserved evidence of inputs, decisions, artifacts, and failures.

The resulting principle is straightforward:

A trustworthy agentic system is not a model with more tools. It is a governed process in which purpose, capability, memory, time, evidence, and authority are explicitly connected.

MCP is an important part of that connection because it provides a standardized boundary between reasoning and the external world. But the protocol acquires institutional meaning only within a larger architecture. The essential pedagogical and managerial shift is therefore from asking whether an AI system can perform a task to asking whether the complete process can be understood, constrained, reconstructed, challenged, and improved.

References

- [1] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). “ReAct: Synergizing Reasoning and Acting in Language Models.” *International Conference on Learning Representations*. <https://arxiv.org/abs/2210.03629>
- [2] Lewis, P. et al. (2020). “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” *Advances in Neural Information Processing Systems*, 33. <https://arxiv.org/abs/2005.11401>

[3] World Wide Web Consortium. (2013). "PROV-O: The PROV Ontology." W3C Recommendation. <https://www.w3.org/TR/prov-o/>

[4] Model Context Protocol. (2025). "Architecture Overview." <https://modelcontextprotocol.io/docs/learn/architecture>

[5] Model Context Protocol. (2025). "Specification: Tools, Resources, and Prompts." <https://modelcontextprotocol.io/specification/2025-11-25>

[6] Model Context Protocol. (2025). "Lifecycle." <https://modelcontextprotocol.io/specification/2025-11-25/basic/lifecycle>

[7] OpenAI. (2026). "Build Your MCP Server." *OpenAI Apps SDK Documentation*. <https://developers.openai.com/apps-sdk/build/mcp-server>

[8] National Institute of Standards and Technology. (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST AI 100-1. <https://doi.org/10.6028/NIST.AI.100-1>

[9] National Institute of Standards and Technology. (2024). *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. NIST AI 600-1. <https://doi.org/10.6028/NIST.AI.600-1>

[10] OpenAI. (2026). "MCP and Connectors." *OpenAI API Documentation*. <https://developers.openai.com/api/docs/guides/tools-connectors-mcp>

[11] OpenAI. (2026). "Authentication." *OpenAI Apps SDK Documentation*. <https://developers.openai.com/apps-sdk/build/auth>

[12] Obsidian. (2026). "Internal Links and Properties." *Obsidian Help*. <https://help.obsidian.md/links> and <https://help.obsidian.md/properties>

Appendix A. Minimal Tool Contract

```
{
  "name": "get_market_snapshot",
  "description": "Return normalized observations for the approved index universe.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "universe_version": {"type": "string"},
      "mode": {"enum": ["SIMULATED", "LIVE", "DELAYED"]},
      "cycle": {"type": "integer", "minimum": 1}
    },
    "required": ["universe_version", "mode", "cycle"]
  }
}
```

The schema constrains shape, not truth. The server must still enforce authorization, validate the universe, retrieve or generate values according to mode, attach provenance, and return explicit errors.

Appendix B. Governed State Machine

State	Entry condition	Permitted next states
CREATED	Valid run configuration exists	AUTHORIZED, REJECTED
AUTHORIZED	Authorization envelope passes	OBSERVED, FAILED

State	Entry condition	Permitted next states
OBSERVED	Snapshot returned	VALIDATED, QUARANTINED
VALIDATED	Mandatory data gates pass	MEMORY_READY
MEMORY_READY	Correct predecessors retrieved	SYNTHESIZED, FAILED
SYNTHESIZED	Draft report exists	PUBLISHED, WITHHELD
PUBLISHED	Persistence and readback succeed	VERIFIED, FAILED
VERIFIED	Integrity and lineage pass	next cycle or FINALIZED
REJECTED / FAILED / WITHHELD	A policy or operational failure occurs	authorized retry, escalation, or termination

Appendix C. Minimum Production Checklist

- ▶ Named accountable owner and documented purpose.
- ▶ Licensed provider and documented data entitlements.
- ▶ Versioned schemas, universe, policies, and prompts.
- ▶ Least-privilege service identities and secret management.
- ▶ Market calendars, exchange time zones, and freshness rules.
- ▶ Idempotent writes and duplicate-trigger protection.
- ▶ Deterministic validation outside the language model.
- ▶ Explicit claim-evidence policy for causal narratives.
- ▶ Append-oriented audit events and independent log retention.
- ▶ Human approval thresholds and emergency stop procedure.
- ▶ Adversarial tests for prompt injection and poisoned memory.
- ▶ Incident response, correction, retention, and deletion procedures.